

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: JavaScript & TypeScript, Manipulación del DOM, Funciones Avanzadas (Closures, IIFE, Arrow), Canvas API y Animaciones

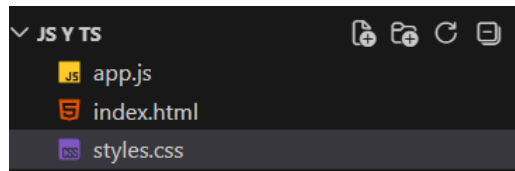
Modalidad: Presencial / Laboratorio

Duración: 90 minutos

APELLIDOS Y NOMBRES: Alarcón Mendoza Estiven Rodrigo

Paso 1: Estructuración y Carga Diferida

Se crearon los archivos base (index.html, styles.css, app.js). Para garantizar que el DOM esté completamente disponible antes de la ejecución de la lógica, se integró el script con el atributo defer. El maquetado se realizó estrictamente en Vanilla JS, sin uso de frameworks externos, preparando los contenedores para el Bento Grid y el Canvas



HTML:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>JumpBot - Blockchain Power</title>
  <link rel="stylesheet" href="styles.css">
  <!-- Carga diferida del script como pide la guía -->
  <script src="app.js" defer></script>
</head>
```

JS:

```
const canvas = document.getElementById('networkCanvas');
const ctx = canvas.getContext('2d');
let animationFrameId;
let lastTime = 0;
let particles = [];

// Constantes físicas para la animación
const PARTICLE_COUNT = 50;
const MAX_DISTANCE = 120;
const SPEED_MULTIPLIER = 0.05;

// Estado del DOM
const bentoCards = document.querySelectorAll('.bento-card');
```

Paso 2: Implementación de IIFE y Closures

Se aplicó el patrón IIFE `((() => { ... }));` para aislar el scope de la aplicación y prevenir la contaminación del objeto global. Mediante closures, se protegieron las variables de estado de la animación (como `particles` y `lastTime`), permitiendo que la función de renderizado acceda a ellas de manera segura y persistente entre cada actualización de frame.

```
((() => {  
  // === CLOSURE SCOPE ===  
  // Las siguientes variables forman parte del closure. Mantienen el  
  estado  
  // de la animación y del DOM protegidos del Global Scope. No se  
  reinician  
  // en cada frame, permitiendo que la animación retenga los datos de  
  las partículas.  
  
  // Estado del Canvas y Animación  
  const canvas = document.getElementById('networkCanvas');  
  const ctx = canvas.getContext('2d');  
  let animationFrameId;  
  let lastTime = 0;  
  let particles = [];
```

```
// Constantes físicas para la animación  
const PARTICLE_COUNT = 50;  
const MAX_DISTANCE = 120;  
const SPEED_MULTIPLIER = 0.05;
```

```
const animate = (timestamp) => {  
  // Cálculo del dt para movimiento uniforme independiente de los  
  FPS  
  if (!lastTime) lastTime = timestamp;  
  const dt = timestamp - lastTime;  
  lastTime = timestamp;
```

Paso 3: Manipulación Dinámica del DOM

Se utilizaron métodos modernos como `querySelectorAll` y `addEventListener` para dotar de interactividad a las tarjetas del Bento Grid. La modificación visual se manejó delegando la responsabilidad a CSS mediante `classList.add()` y `classList.remove()`, evitando la inyección de estilos en línea y optimizando el proceso de reflow/repaint del navegador.

```
/* Features - Bento Grid & Glassmorphism */  
.features {  
  padding: 80px 5%;  
  text-align: center;  
}
```

```
.section-header {
  margin-bottom: 50px;
}

.section-header h2 {
  font-size: 2.5rem;
  margin-bottom: 15px;
}

.section-header p {
  color: var(--text-muted);
}

.bento-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  max-width: 1200px;
  margin: 0 auto;
}

.bento-card {
  background: var(--glass-bg);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid var(--glass-border);
  border-radius: 20px;
  padding: 30px;
  text-align: left;
  transition: transform 0.3s ease, border-color 0.3s ease;
}

.bento-card:hover {
  transform: translateY(-5px);
  border-color: rgba(0, 210, 255, 0.3);
}

.bento-card .icon {
  font-size: 2rem;
  margin-bottom: 20px;
  background: linear-gradient(135deg, rgba(58, 123, 213, 0.2), rgba(0, 210, 255, 0.05));
  width: 60px;
  height: 60px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 12px;
```

```

    border: 1px solid rgba(0, 210, 255, 0.1);
  }

.bento-card h3 {
  margin-bottom: 10px;
  font-size: 1.2rem;
}

.bento-card p {
  color: var(--text-muted);
  font-size: 0.9rem;
  line-height: 1.5;
}

/* Clase dinámica para JS */
.card-active {
  box-shadow: 0 0 20px rgba(0, 210, 255, 0.15);
  border-color: var(--primary-blue);
}

```

Paso 4: Animación Interactiva (Canvas 2D)

Se desarrolló una animación de partículas interconectadas simulando una red. En lugar de usar `setInterval`, se implementó `requestAnimationFrame` acoplado al cálculo de Delta Time (dt). Esto garantiza que la velocidad de las partículas sea constante y matemáticamente uniforme, independientemente de la tasa de refresco (FPS) del dispositivo.

```

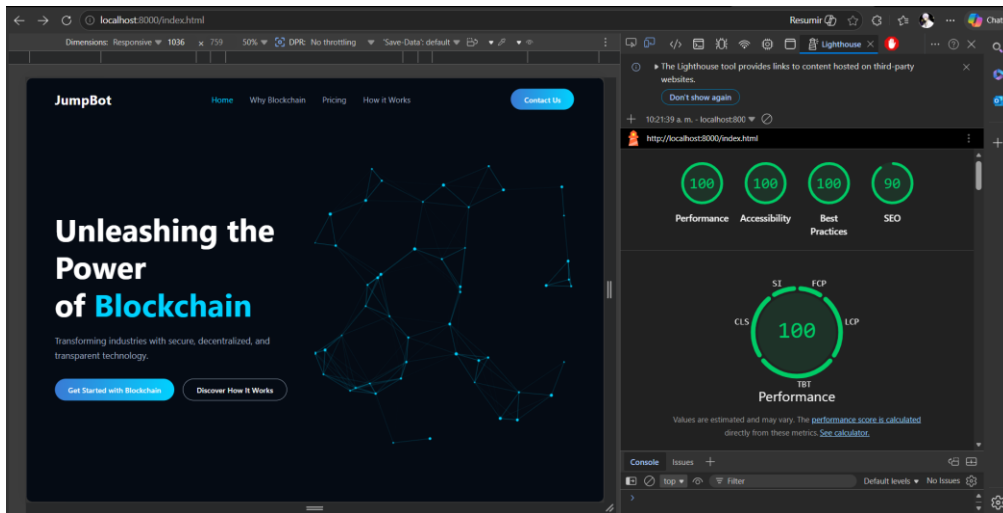
// === INICIALIZACIÓN ===
const initApp = () => {
  initCanvas();
  setupDOMEvents();
  // Iniciar el render pipeline
  animationFrameId = requestAnimationFrame(animate);
};

// Ejecución segura una vez que el DOM está listo
document.addEventListener('DOMContentLoaded', initApp);

```

Paso 5: Depuración y Optimización

Se emplearon las DevTools (pestaña Performance y Memory) para auditar la ejecución. Se aseguró la recolección de basura (Garbage Collection) manejando correctamente el evento `resize` de la ventana, donde se invoca `cancelAnimationFrame` para destruir el bucle anterior antes de redimensionar el canvas, previniendo fugas de memoria y nodos huérfanos.



RESULTADO: [Ichimaru465/Js-y-Ts](https://github.com/ichimaru465/Js-y-Ts)

